RankScript

Competitive Learning Platform

Technical Documentation

Version 1.0 | 2026

Technologies

FastAPI | Next.js | PostgreSQL | Redis | Docker | SQLAlchemy | Tailwind CSS

1. Project Overview

1.1 What is RankScript?

RankScript is a full-stack competitive learning platform. Students learn from mentor-created courses, take quizzes, and submit assignments. The system tracks performance and ranks students on a live leaderboard.

1.2 Problem It Solves

- Standard e-learning platforms have no competition or ranking.
- Students have no motivation to perform better than peers.
- Mentors have no tool to track student performance across all activities.
- Admins cannot approve and moderate content on one platform.

1.3 Target Users

Role	Capabilities	Dashboard
Student	Enroll, learn, take quizzes, submit assignments, view rank	/student/dashboard
Mentor	Create courses, add lessons and quizzes, grade submissions, view analytics	/mentor/dashboard
Admin	Approve courses and lessons, manage users, view platform analytics, audit logs	/admin/dashboard

2. System Architecture

2.1 High-Level Architecture

RankScript uses a four-layer architecture separated into independent Docker containers.

```
Browser (port 3000)
     |
Next.js Frontend  [Container: rankscript_frontend]
     | /api/* rewrite
FastAPI Backend   [Container: rankscript_backend  :8000]
     |                    |
PostgreSQL DB     [Container: rankscript_db       :5432]
Redis Cache       [Container: rankscript_redis    :6379]
```

2.2 Request Flow

- 1. Browser sends a request to Next.js on port 3000 (e.g. POST /api/auth/login).
- 2. Next.js rewrite rule strips /api prefix and forwards to FastAPI on port 8000.
- 3. FastAPI validates the JWT token using the Bearer scheme in deps.py.
- 4. The route handler calls the appropriate service function (e.g. login_user).
- 5. The service queries PostgreSQL via SQLAlchemy ORM.
- 6. The result is serialized by a Pydantic schema and returned as JSON.
- 7. Next.js forwards the JSON to the browser.

2.3 Key Design Principles

- Separation of Concerns: Routes only handle HTTP. Business logic lives in services.
- Schema Validation: All input and output is validated by Pydantic schemas.
- Role-Based Access: deps.py enforces student, mentor, and admin roles on every route.
- Docker Isolation: Each service runs in its own container on a shared bridge network.
- No CORS from Browser: Next.js rewrites proxy all API calls server-side, so the browser never contacts port 8000 directly.

3. Technologies Used

3.1 FastAPI (Backend Framework)

- FastAPI is a modern Python web framework for building APIs.
- It uses Python type hints to automatically validate request and response data.
- It generates interactive API documentation at /docs automatically.
- It is asynchronous by design, making it fast under high load.
- In RankScript: FastAPI handles all backend routes — auth, courses, quizzes, assignments, rankings, and analytics.
- File: backend/app/main.py is the FastAPI application entry point.

3.2 Next.js (Frontend Framework)

- Next.js is a React framework with server-side rendering support.
- It supports file-based routing — each page.tsx file becomes a URL route.
- It has a built-in rewrite engine used to proxy /api/* to the FastAPI backend.
- It is deployed as a standalone Node.js server inside Docker.
- In RankScript: All student, mentor, and admin pages are Next.js pages.
- Key config: frontend/next.config.js defines the /api rewrite rule.

3.3 PostgreSQL (Database)

- PostgreSQL is an open-source relational database.
- It supports UUIDs, ENUM types, JSON columns, triggers, and constraints natively.
- It stores all platform data: users, courses, lessons, quizzes, assignments, submissions, and rankings.
- In RankScript: 12 tables are defined in database/00_schema.sql.
- Two database triggers auto-update total_enrolled and total_lessons counts.

3.4 Redis

- Redis is an in-memory key-value data store.
- It is used for caching and session management in production systems.
- It is extremely fast because all data is stored in RAM.
- In RankScript: Redis is configured and running as a container, ready for token blacklisting and caching.
- Config: redis/redis.conf sets persistence and memory options.

3.5 Docker

- Docker packages applications into containers with all their dependencies.
- docker-compose.yml defines and orchestrates all five services together.

- Each service has its own Dockerfile with build instructions.
- In RankScript: Docker runs the backend, frontend, PostgreSQL, Redis, and optional pgAdmin.
- All services communicate via the rankscript_network Docker bridge network.

3.6 SQLAlchemy (ORM)

- SQLAlchemy is a Python library that maps Python classes to database tables.
- ORM means Object-Relational Mapper — you write Python, not SQL.
- It handles connection pooling, transactions, and relationship joins automatically.
- In RankScript: All models (User, Course, Lesson, etc.) are SQLAlchemy classes in backend/app/models/.
- Session management is done in backend/app/db/session.py.

3.7 Pydantic

- Pydantic is a Python data validation library using type annotations.
- It validates incoming request bodies and serializes outgoing responses.
- If data is invalid, it returns a clear 422 error with the field name and reason.
- In RankScript: All schemas in backend/app/schemas/ are Pydantic models.
- Example: SubmissionCreate validates that content or file_url must be present.

3.8 React Context (AuthContext)

- React Context is a built-in React feature for sharing state across components.
- It avoids prop drilling by providing data to any component in the tree.
- In RankScript: AuthContext.tsx stores the logged-in user, login/logout functions, and token refresh logic.
- All components access user data through the useAuth() hook.
- Tokens are stored in browser cookies using the js-cookie library.

3.9 Axios

- Axios is a JavaScript library for making HTTP requests from the browser.
- It supports interceptors — functions that run before every request or response.
- In RankScript: axiosInstance.ts creates a configured Axios instance.
- The request interceptor automatically attaches the Bearer token to every API call.
- The response interceptor auto-refreshes expired access tokens using the refresh token.

3.10 Tailwind CSS

- Tailwind CSS is a utility-first CSS framework.
- Instead of writing CSS files, you apply pre-defined class names in JSX.

- Example: bg-gray-900 sets background color, rounded-2xl adds border radius.
- In RankScript: All UI components use Tailwind classes. No custom CSS files are needed.
- Config: frontend/tailwind.config.js sets the content paths.

3.11 Pytest (Backend Testing)

- Pytest is the standard Python testing framework.
- It auto-discovers test files named test_*.py and runs all functions starting with test_.
- In RankScript: Tests are in backend/tests/. Covers auth, courses, quizzes, assignments, analytics, and rankings.
- Config: backend/tests/conftest.py sets up a test database and client fixtures.

3.12 Vitest (Frontend Testing)

- Vitest is a fast unit testing framework for JavaScript/TypeScript.
- It is compatible with Jest's API but runs inside Vite's build pipeline.
- In RankScript: Tests are in frontend/src/test/ and frontend/src/services/*.test.ts.
- Config: frontend/vitest.config.ts sets jsdom as the test environment.

4. Project Structure

4.1 Root Level

Folder / File	Purpose
backend/	FastAPI Python application. All API logic lives here.
frontend/	Next.js TypeScript application. All UI pages live here.
database/	SQL schema and seed data files. Loaded by PostgreSQL on first start.
redis/	Redis configuration file (redis.conf).
docker-compose.yml	Defines and connects all Docker services.
.env	Environment variables: DB credentials, JWT secret, API URLs.

4.2 Backend Structure (backend/)

Path	Purpose
app/main.py	FastAPI app entry point. Registers middleware, exception handlers, and all routers.
app/core/config.py	Loads environment variables into a Settings object using pydantic-settings.
app/core/security.py	Password hashing with bcrypt. JWT token creation and decoding.
app/db/session.py	Creates the SQLAlchemy engine and session factory. Provides get_db() dependency.
app/db/base.py	Defines the SQLAlchemy Base class that all models inherit from.
app/api/deps.py	FastAPI dependencies: get_current_user(), get_admin(), get_mentor(), get_student().
app/api/routes/	One file per resource: auth, users, courses, lessons, enrollments, quizzes, assignments, ranking, analytics, admin.
app/services/	Business logic. Services are called by routes. They query the database and apply rules.
app/models/	SQLAlchemy ORM models. Each model maps to a database table.
app/schemas/	Pydantic schemas for request validation and response serialization.
tests/	Pytest test files. Covers all major API endpoints and services.

4.3 Frontend Structure (frontend/src/)

Path	Purpose
app/	Next.js App Router pages. Each folder is a URL route.
app/auth/login/	Login page. Calls AuthContext.login().
app/auth/register/	Registration page.
app/student/dashboard/	Student dashboard: enrolled courses, rank, analytics.
app/mentor/dashboard/	Mentor dashboard: course management, analytics.
app/admin/dashboard/	Admin dashboard: platform stats, course approvals.
app/courses/	Course listing and detail pages. Assignment and quiz sub-pages.
app/leaderboard/	Public leaderboard page.
services/	API call functions grouped by resource (auth, course, assignment, etc.).
context/AuthContext.tsx	Global auth state. Provides user, login, logout, refresh to all components.
hooks/useAuth.ts	Shorthand hook that returns useAuthContext().
lib/axiosInstance.ts	Configured Axios instance with auth token interceptors.
components/	Reusable UI components: Navbar, CourseCard, Button, Input, Toast, QuizQuestion.

5. File-Level Deep Explanation

5.1 Backend — Core Files

backend/app/main.py

Entry point of the FastAPI application. Configures CORS, exception handlers, and registers all route modules.

- startup_event() — Creates all database tables on app startup using SQLAlchemy.
- preflight_handler() — Fallback OPTIONS handler to prevent 405 errors on CORS preflight.
- validation_exception_handler() — Converts Pydantic 422 errors to clean JSON.
- http_exception_handler() — Converts HTTPException to JSONResponse so CORS headers are preserved.
- global_exception_handler() — Catches all unhandled exceptions and returns a 500 JSON response.
- root() — Returns API health status at GET /.
- health_check() — Returns {status: healthy} at GET /health.
- debug_cors() — Returns CORS debug info at GET /debug/cors.

backend/app/api/deps.py

Authentication and authorization dependencies injected into route handlers.

- get_current_user() — Validates Bearer JWT token. Returns authenticated User object.
- require_role(*roles) — Factory function. Returns a dependency that checks user role.
- get_admin() — Requires admin role. Used on admin-only routes.
- get_mentor() — Requires mentor or admin role.
- get_student() — Requires student role.
- get_student_or_admin() — Requires student or admin role.

backend/app/core/security.py

Handles password security and JWT token operations.

- hash_password(password) — Hashes plain text password using bcrypt with 12 rounds.
- verify_password(plain, hashed) — Compares plain text against bcrypt hash. Returns True/False.
- create_access_token(data) — Creates a short-lived JWT (30 minutes) with type=access.
- create_refresh_token(data) — Creates a long-lived JWT (7 days) with type=refresh.
- decode_token(token) — Decodes and verifies a JWT. Returns payload dict or None if invalid.

backend/app/core/config.py

Loads all environment variables into a typed Settings object using pydantic-settings.

- Settings class — Defines DATABASE_URL, REDIS_URL, SECRET_KEY, ALGORITHM, ACCESS_TOKEN_EXPIRE_MINUTES, REFRESH_TOKEN_EXPIRE_DAYS, DEBUG, ENVIRONMENT, ALLOWED_ORIGINS.

backend/app/db/session.py

Manages the SQLAlchemy database connection pool.

- engine — SQLAlchemy engine with pool_size=10, max_overflow=20. Supports PostgreSQL and SQLite.
- SessionLocal — Session factory bound to the engine.
- get_db() — FastAPI dependency generator. Yields a DB session and closes it after the request.

5.2 Backend — Routes

backend/app/api/routes/auth.py

- register() — POST /auth/register. Creates a new user account.
- login() — POST /auth/login. Validates credentials. Returns access and refresh tokens.
- refresh() — POST /auth/refresh. Issues a new access token from a valid refresh token.

backend/app/api/routes/users.py

- get_me() — GET /users/me. Returns the current authenticated user's profile.
- update_me() — PUT /users/me. Updates name, state, district, bio.
- list_users() — GET /users. Returns all users. Admin only.
- get_user() — GET /users/{user_id}. Returns a specific user by ID.

backend/app/api/routes/courses.py

- list_courses() — GET /courses. Returns all approved courses with pagination.
- list_pending() — GET /courses/pending. Returns courses pending admin approval.
- list_pending_lessons() — GET /courses/pending-lessons. Returns lessons awaiting admin review.
- my_courses() — GET /courses/my. Returns all courses created by the current mentor.
- create() — POST /courses. Creates a new course in pending status. Mentor only.
- get_course() — GET /courses/{course_id}. Returns one course by ID.
- update() — PUT /courses/{course_id}. Updates course fields. Mentor only.
- approve() — PUT /courses/{course_id}/approve. Approves or rejects a course. Admin only.
- get_lessons_for_review() — GET /courses/{course_id}/lessons-for-review. Admin only.
- review_lesson_endpoint() — PUT /courses/lessons/{lesson_id}/review. Admin approves/rejects a lesson.
- delete_course_endpoint() — DELETE /courses/{course_id}. Deletes course and all related data.
- check_enrollment_count() — GET /courses/{course_id}/enrollment-count. Returns enrolled student count.

backend/app/api/routes/lessons.py

- list_lessons() — GET /courses/{course_id}/lessons. Returns all lessons for a course.
- create_lesson() — POST /courses/{course_id}/lessons. Adds a lesson. Mentor only.
- remove_lesson() — DELETE /courses/{course_id}/lessons/{lesson_id}. Removes a lesson.
- check_lesson_enrollment_count() — GET .../{lesson_id}/enrollment-count. Returns parent course enrollment count.

backend/app/api/routes/enrollments.py

- enroll() — POST /enrollments. Enrolls the current student in a course.
- my_enrollments() — GET /enrollments/me. Returns all enrollments for the current student.
- track_progress() — PUT /enrollments/{enrollment_id}/progress. Updates lesson progress percentage.

backend/app/api/routes/quizzes.py

- list_quizzes() — GET /courses/{course_id}/quizzes. Returns all active quizzes for a course.
- create() — POST /courses/{course_id}/quizzes. Creates a quiz. Mentor only.
- get_quiz() — GET /courses/{course_id}/quizzes/{quiz_id}. Returns one quiz.
- add_question_to_quiz() — POST .../questions. Adds a question to a quiz.
- list_questions() — GET .../questions. Returns all questions for a quiz.
- attempt_quiz() — POST .../attempt. Submits quiz answers. Calculates score. Updates ranking.
- my_attempts() — GET .../attempts/me. Returns current student's attempts for a quiz.

backend/app/api/routes/assignments.py

- list_assignments() — GET /courses/{course_id}/assignments. Returns assignments for a course.
- create() — POST /courses/{course_id}/assignments. Creates an assignment. Mentor only.
- submit() — POST .../submit. Student submits a text answer. Validates enrollment and deadline.
- my_submission() — GET .../submission/me. Returns current student's submission.
- all_submissions() — GET .../submissions. Returns all submissions. Mentor only.
- grade() — PUT .../submissions/{submission_id}/grade. Mentor grades a submission. Triggers ranking update.

backend/app/api/routes/ranking.py

- build_leaderboard() — Helper function. Formats rank entries into leaderboard response with is_me flag.
- indian_leaderboard() — GET /rankings/global. Returns top students across India.
- state_leaderboard() — GET /rankings/state/{state}. Returns top students in a state.
- district_leaderboard() — GET /rankings/district/{district}. Returns top students in a district.
- my_rank() — GET /rankings/me. Returns the current student's rank at all levels.
- recalculate_my_rank() — POST /rankings/recalculate. Recalculates the current user's rank score.

- recalculate_user_rank() — POST /rankings/recalculate/{user_id}. Admin recalculates rank for any user.

backend/app/api/routes/analytics.py

- student_analytics() — GET /analytics/student/me. Returns personal stats for the current student.
- mentor_analytics() — GET /analytics/mentor/overview. Returns stats for mentor's courses.
- admin_analytics() — GET /analytics/admin/overview. Returns platform-wide statistics.

backend/app/api/routes/admin.py

- generate_reference_id() — Generates unique audit reference ID (e.g. AUD-20260101-ABCD1234).
- get_admin_leaderboard() — GET /admin/leaderboard. Paginated user list with filtering.
- search_users() — GET /admin/search. Searches users by name or email.
- get_geographic_filters() — GET /admin/geographic-filters. Returns available states and districts.
- get_districts_for_state() — GET /admin/districts-for-state. Returns districts within a state.
- get_geographic_stats() — GET /admin/geographic-stats. Returns stats for a region.
- get_user_detail() — GET /admin/user/{user_id}. Returns detailed user profile.
- remove_student() — DELETE /admin/remove-student/{user_id}. Removes a student with confirmation.
- get_mentors_for_reassignment() — GET /admin/mentors-for-reassignment. Lists mentors for reassignment.
- remove_mentor() — DELETE /admin/remove-mentor/{user_id}. Removes mentor and optionally reassigns students.
- get_audit_log() — GET /admin/audit-log. Returns paginated audit log.

5.3 Backend — Services

backend/app/services/auth_service.py

- register_user(db, data) — Checks for duplicate email. Creates User with hashed password. Saves to DB.
- login_user(db, data) — Finds user by email. Verifies password. Updates last_login. Returns JWT tokens.
- refresh_access_token(refresh_token) — Decodes refresh token. Issues new access token. Returns original refresh token unchanged.

backend/app/services/course_service.py

- create_course(db, data, mentor) — Creates a course in pending status.
- get_courses(db, status, skip, limit) — Returns filtered, paginated courses.
- get_course_by_id(db, course_id) — Returns a course or raises 404.

- update_course(db, course_id, data, mentor) — Updates course fields if caller is the mentor or admin.
- approve_course(db, course_id, data, admin) — Sets course status to approved or rejected. Records reviewer.
- get_mentor_courses(db, mentor_id) — Returns all courses for a mentor.
- get_pending_courses(db) — Returns all courses with status=pending.
- get_pending_lessons(db) — Returns all lessons with review_status=pending.
- review_lesson(db, lesson_id, data, admin) — Approves or rejects a lesson. Records feedback.
- get_course_lessons_for_review(db, course_id) — Returns all lessons for a course.
- delete_course(db, course_id, mentor) — Deletes course and all related data (lessons, quizzes, assignments, enrollments).
- get_course_enrollment_count(db, course_id) — Returns enrollment count for a course.

backend/app/services/assignment_service.py

- create_assignment(db, course_id, data, mentor) — Creates an assignment record.
- get_assignments_for_course(db, course_id) — Returns active assignments for a course.
- get_assignment_by_id(db, assignment_id) — Returns assignment or raises 404.
- submit_assignment(db, assignment_id, data, student) — Validates enrollment and deadline. Creates Submission. Handles late penalty calculation.
- grade_submission(db, submission_id, data, mentor) — Sets score and feedback. Applies late penalty. Triggers ranking update.
- get_my_submission(db, assignment_id, student_id) — Returns student's submission for an assignment.
- get_all_submissions(db, assignment_id) — Returns all submissions for an assignment.
- get_course_submissions(db, course_id) — JOIN query to get all submissions across all assignments in a course.

backend/app/services/enrollment_service.py

- enroll_student(db, data, student) — Checks course availability and existing enrollment. Creates Enrollment. Auto-approves if course is not gated.
- get_my_enrollments(db, student_id) — Returns all enrollments for a student.
- update_progress(db, enrollment_id, data, student) — Updates progress percentage. Marks completed if 100%. Triggers ranking update.

backend/app/services/quiz_service.py

- create_quiz(db, course_id, data, mentor) — Creates a Quiz record.
- get_quizzes_for_course(db, course_id) — Returns active quizzes with question count.
- get_quiz_by_id(db, quiz_id) — Returns quiz or raises 404.
- add_question(db, quiz_id, data, mentor) — Adds a question to a quiz.
- get_questions(db, quiz_id) — Returns questions ordered by position.
- submit_quiz(db, quiz_id, data, student) — Validates enrollment and attempt limit. Scores answers. Creates QuizAttempt. Triggers ranking update.

- get_my_attempts(db, quiz_id, student_id) — Returns student's attempts sorted by date.
- get_best_score(db, quiz_id, student_id) — Returns highest score across all attempts.

backend/app/services/ranking_service.py

Implements the weighted ranking formula. All score components are 0-100 scale.

- get_or_create_rank_entry(db, user) — Gets or creates a RankEntry row for the user.
- calculate_quiz_score(db, user_id) — Averages the best score per quiz across all quizzes attempted.
- calculate_assignment_score(db, user_id) — Averages score across all graded submissions.
- calculate_completion_score(db, user_id) — Averages progress percentage across all approved enrollments.
- calculate_streak_score(streak_days) — Converts consecutive active days to score (caps at 30 days = 100).
- compute_rank_score(quiz, assignment, completion, streak) — Final score = Quiz*0.40 + Assignment*0.30 + Completion*0.15 + Streak*0.15.
- update_user_ranking(db, user) — Recalculates all component scores and saves updated RankEntry.
- update_streak(db, user) — Increments streak if active yesterday, resets if gap > 1 day.
- get_indian_leaderboard(db, skip, limit) — Returns top students by rank score across all of India.
- get_state_leaderboard(db, state, skip, limit) — Returns top students in a given state.
- get_district_leaderboard(db, district, skip, limit) — Returns top students in a given district.
- get_my_ranks(db, user) — Returns rank positions at national, state, and district levels.

backend/app/services/analytics_service.py

- get_student_analytics(db, user) — Aggregates enrollment counts, quiz scores, assignment scores, streak, and rank.
- get_mentor_analytics(db, mentor) — Aggregates stats across all mentor's courses: enrolled students, completion rates, pending submissions.
- get_admin_analytics(db) — Aggregates platform-wide totals: users, courses, enrollments, attempts, top 10 students.

5.4 Backend — Models

Models Summary Table

Model	Table	Key Fields
User	users	id, name, email, password_hash, role, xp, rank_score, streak_days
Course	courses	id, mentor_id, title, level, status, is_gated, total_lessons, total_enrolled

Lesson	lessons	id, course_id, title, youtube_url, youtube_id, order, is_free, review_status
Enrollment	enrollments	id, student_id, course_id, progress, lessons_done, is_approved, is_completed
Quiz	quizzes	id, course_id, mentor_id, title, time_limit, pass_score, max_attempts
Question	questions	id, quiz_id, text, option_a-d, correct_option, points, order
QuizAttempt	quiz_attempts	id, quiz_id, student_id, answers(JSON), score, passed, time_taken
Assignment	assignments	id, course_id, mentor_id, title, max_score, deadline, late_penalty, allow_late
Submission	submissions	id, assignment_id, student_id, content, score, is_graded, is_late, late_days
RankEntry	rank_entries	id, user_id, quiz_score, assignment_score, completion_score, streak_score, rank_score
AuditLog	audit_logs	id, admin_id, action, target_user_id, reference_id, details

5.5 Frontend — Services

frontend/src/services/authService.ts

- loginUser(data) — POST /api/auth/login. Returns access and refresh tokens.
- registerUser(data) — POST /api/auth/register. Creates a new user account.
- refreshToken(token) — POST /api/auth/refresh. Returns new access token.
- getMe() — GET /api/users/me. Returns current user profile using axiosInstance.

frontend/src/services/courseService.ts

- getCourses() — GET /api/courses. Returns approved courses list and total count.
- getCourse(id) — GET /api/courses/{id}. Returns a single course.
- createCourse(data) — POST /api/courses. Creates a new course.
- getMyCourses() — GET /api/courses/my. Returns mentor's own courses.
- getPendingCourses() — GET /api/courses/pending. Returns courses awaiting approval.
- updateCourseStatus(id, status) — PUT /api/courses/{id}/approve. Admin approves or rejects.

- addLesson(courseId, data) — POST /api/courses/{courseId}/lessons. Adds a lesson.
- getLessons(courseId) — GET /api/courses/{courseId}/lessons. Returns all lessons.
- getPendingLessons() — GET /api/courses/pending-lessons. Returns lessons pending review.
- reviewLesson(lessonId, status, feedback) — PUT /api/courses/lessons/{id}/review. Admin reviews lesson.
- deleteCourse(courseId) — DELETE /api/courses/{courseId}. Deletes a course.
- getCourseEnrollmentCount(courseId) — Returns enrollment count for a course.
- deleteLesson(courseId, lessonId) — Deletes a lesson from a course.
- getLessonEnrollmentCount(courseId, lessonId) — Returns enrollment count for a lesson's course.

frontend/src/services/assignmentService.ts

- getAssignments(courseId) — Returns all assignments for a course.
- createAssignment(courseId, data) — Creates an assignment.
- submitAssignment(courseId, assignmentId, content) — Posts student's text submission.
- getMySubmission(courseId, assignmentId) — Returns student's own submission. Returns null on 404.
- getAllSubmissions(courseId, assignmentId) — Returns all submissions (mentor view).
- gradeSubmission(courseId, assignmentId, submissionId, score, feedback) — Grades a submission.

frontend/src/services/enrollmentService.ts

- enrollInCourse(courseId) — POST /api/enrollments. Enrolls the current student.
- getMyEnrollments() — GET /api/enrollments/me. Returns all student enrollments.
- updateProgress(enrollmentId, data) — PUT /api/enrollments/{id}/progress. Updates lesson progress.

frontend/src/services/quizService.ts

- getQuizzes(courseId) — Returns all quizzes for a course.
- createQuiz(courseId, data) — Creates a quiz.
- getQuestions(courseId, quizId) — Returns quiz questions (without correct answers for students).
- addQuestion(courseId, quizId, data) — Adds a question to a quiz.
- submitQuiz(courseId, quizId, answers, timeTaken) — Submits quiz answers and returns scored result.
- getMyAttempts(courseId, quizId) — Returns student's past attempts.

frontend/src/services/rankingService.ts

- getIndianLeaderboard(skip, limit) — Returns national leaderboard.
- getStateLeaderboard(state, skip, limit) — Returns state leaderboard.
- getDistrictLeaderboard(district, skip, limit) — Returns district leaderboard.

- getMyRank() — Returns current student's rank at all levels.
- recalculateMyRank() — Triggers backend to recalculate the current user's rank.

frontend/src/services/analyticsService.ts

- getStudentAnalytics() — Returns personal stats for the current student.
- getMentorAnalytics() — Returns stats for the mentor's courses.
- getAdminAnalytics() — Returns platform-wide statistics for the admin.

frontend/src/services/adminService.ts

- getAdminLeaderboard(params) — Returns paginated user list with geographic filters.
- searchUsers(query, role) — Searches users by name or email.
- getGeographicFilters() — Returns available states and districts for filter dropdowns.
- getDistrictsForState(state) — Returns districts within a specific state.
- getGeographicStats(params) — Returns aggregated stats for a geographic region.
- getUserDetail(userId) — Returns detailed profile for one user.
- removeStudent(userId, data) — Removes a student with name confirmation and admin password.
- getMentorsForReassignment() — Returns list of mentors for student reassignment.
- removeMentor(userId, data) — Removes a mentor. Optionally reassigns their students.
- getAuditLog(params) — Returns paginated admin action audit log.

5.6 Frontend — Context and Hooks

frontend/src/context/AuthContext.tsx

- fetchMe(token) — Calls GET /api/users/me with Bearer token. Returns User or null.
- login(email, password) — Posts credentials. Saves tokens to cookies. Fetches user profile. Redirects by role.
- register(data) — Posts registration data. Redirects to login page.
- logout() — Clears cookies. Resets user state. Redirects to login page.
- refresh() — Uses refresh token cookie to get new access token. Updates cookie.

frontend/src/lib/axiosInstance.ts

- api (Axios instance) — Base URL is /api. Sends Content-Type: application/json on all requests.
- Request Interceptor — Reads access_token from cookies. Adds Authorization: Bearer header.
- Response Interceptor — On 401, tries to refresh token automatically. Retries original request. On failure, redirects to login.

frontend/src/hooks/useAuth.ts

- useAuth() — Returns the AuthContext value (user, login, logout, etc.). Shorthand for useAuthContext().

6. Data Flow Explanation

6.1 Login Flow

- 1. User enters email and password on /auth/login page.
- 2. Login page calls AuthContext.login(email, password).
- 3. AuthContext sends POST /api/auth/login to Next.js server.
- 4. Next.js rewrite forwards the request to FastAPI at http://backend:8000/auth/login.
- 5. FastAPI calls login_user() in auth_service.py.
- 6. login_user() queries users table for the email.
- 7. verify_password() checks bcrypt hash against submitted password.
- 8. If valid, last_login is updated. Streak is updated for students.
- 9. create_access_token() and create_refresh_token() generate two JWTs.
- 10. Tokens are returned to the frontend as JSON.
- 11. AuthContext saves tokens to browser cookies.
- 12. AuthContext calls fetchMe() to load the user profile.
- 13. User is redirected to the appropriate dashboard by role.

6.2 Course Creation Flow

- 1. Mentor fills in the create course form at /mentor/courses/create.
- 2. Frontend calls createCourse(data) in courseService.ts.
- 3. POST /api/courses is sent with the Bearer token header.
- 4. FastAPI get_mentor() dependency validates the token and confirms mentor role.
- 5. create_course() in course_service.py creates a Course with status=pending.
- 6. The mentor adds lessons (POST /courses/{id}/lessons) and quizzes.
- 7. Admin sees the course in the pending list at /admin/approvals.
- 8. Admin calls approve_course() which sets status=approved and records reviewed_by.
- 9. The course is now visible to students on the /courses page.

6.3 Assignment Submission Flow

- 1. Student navigates to a course assignment page.
- 2. Frontend calls getAssignments(courseId) to load assignment details.
- 3. Student types their answer in the textarea.
- 4. Student clicks Submit Assignment.
- 5. Frontend calls submitAssignment(courseId, assignmentId, content).
- 6. POST /api/courses/{courseId}/assignments/{assignmentId}/submit is sent.
- 7. Backend verifies student is enrolled and enrollment is approved.
- 8. Backend checks no existing submission exists.
- 9. Backend checks if deadline has passed and calculates late_days if needed.
- 10. Submission record is saved to the submissions table.
- 11. Success response is returned. Frontend shows the submitted content.

- 12. Mentor later grades the submission. Grade triggers ranking recalculation.

6.4 Leaderboard Calculation Flow

- 1. Ranking is recalculated automatically after: quiz submission, assignment grading, and enrollment progress updates.
- 2. update_user_ranking(db, user) is called in ranking_service.py.
- 3. calculate_quiz_score() finds best score per quiz across all attempts. Averages them.
- 4. calculate_assignment_score() averages all graded submission scores.
- 5. calculate_completion_score() averages progress percentage across approved enrollments.
- 6. calculate_streak_score() converts streak_days to a 0-100 score capped at 30 days.
- 7. compute_rank_score() applies the formula: Quiz*0.40 + Assignment*0.30 + Completion*0.15 + Streak*0.15.
- 8. RankEntry row is updated. User.rank_score and User.xp are also updated.
- 9. Students view their rank at /leaderboard or on their dashboard.

7. Features

7.1 Authentication System

- Email and password registration with role selection (student, mentor, admin).
- JWT-based login. Access token expires in 30 minutes. Refresh token valid for 7 days.
- Automatic token refresh — expired tokens are silently refreshed without logging out.
- Tokens stored in browser cookies. Cleared on logout.

7.2 Role-Based Dashboards

- Student Dashboard — Shows enrolled courses, rank score, XP, streak, analytics, and quick links.
- Mentor Dashboard — Shows created courses, total students, pending submissions, analytics.
- Admin Dashboard — Shows platform stats, pending course approvals, user management tools.

7.3 Course Management

- Mentors create courses with title, description, level (beginner/intermediate/advanced), and tags.
- Courses start in draft/pending status. Admin must approve before students can enroll.
- Mentors add YouTube video lessons. Each lesson goes through admin review.
- Gated courses require mentor approval of each student enrollment request.
- Course deletion removes all related lessons, quizzes, assignments, and enrollments.

7.4 Assignment Handling

- Mentors create text-based assignments with max score, passing score, deadline, and late penalty.
- Students submit text answers. One submission per assignment per student.
- Late submissions are accepted if allow_late is enabled. Penalty deducted per day late.
- Mentors grade submissions by entering a score and optional feedback.
- Grading triggers an immediate ranking recalculation.

7.5 Quiz System

- Mentors create multiple-choice quizzes with time limits and pass score thresholds.
- Questions have options A, B, C, D. Correct answer is hidden from student view.
- Maximum attempts limit per quiz is configurable.
- Scores are calculated server-side. Rankings are updated after each attempt.

7.6 Leaderboard System

- Three-tier leaderboard: National (India-wide), State-level, and District-level.
- Each leaderboard entry shows rank, name, score, XP, and location.
- The current student's row is highlighted with is_me = true.
- Ranking formula: 40% Quiz + 30% Assignment + 15% Completion + 15% Streak.

7.7 Admin Approval System

- All new courses enter pending status. Admin sees them in the approvals queue.
- Admin can approve or reject with notes. Rejection reason is stored.
- Individual lessons require admin approval before students can view them.
- Admin can remove students or mentors with a confirmation step and audit trail.
- Every admin action is logged in the audit_logs table with a unique reference ID.

8. Database Design

8.1 All Tables

Table	Purpose	Primary Key
users	All platform users (students, mentors, admins)	id (UUID)
courses	Courses created by mentors	id (UUID)
lessons	YouTube video lessons within courses	id (UUID)
enrollments	Student-course enrollment records	id (UUID)
quizzes	Quizzes linked to courses	id (UUID)
questions	MCQ questions within quizzes	id (UUID)
quiz_attempts	Student quiz submissions and scores	id (UUID)
assignments	Text assignments linked to courses	id (UUID)
submissions	Student assignment submissions	id (UUID)
rank_entries	Per-student ranking scores. One row per student.	id (UUID)
audit_logs	Immutable log of all admin actions	id (UUID)
schema_migrations	Database version tracking	version (INTEGER)

8.2 Key Relationships

- users 1-to-N courses — One mentor creates many courses.
- courses 1-to-N lessons — One course has many lessons.
- courses 1-to-N quizzes — One course has many quizzes.
- courses 1-to-N assignments — One course has many assignments.
- users N-to-N courses (via enrollments) — Many students enroll in many courses.
- quizzes 1-to-N questions — One quiz has many questions.
- quizzes 1-to-N quiz_attempts — One quiz has many student attempts.
- assignments 1-to-N submissions — One assignment has many student submissions.
- users 1-to-1 rank_entries — One student has exactly one ranking row.

8.3 Database Triggers

- trg_enrollment_count — Fires after INSERT/DELETE on enrollments. Auto-updates total_enrolled in the courses table. No manual count queries needed.
- trg_lesson_count — Fires after INSERT/DELETE on lessons. Auto-updates total_lessons in the courses table.

- updated_at triggers — Auto-update the updated_at timestamp column on any row change in users, courses, lessons, enrollments, quizzes, assignments, and rank_entries.

8.4 Enum Types

- user_role — Values: admin, mentor, student.
- course_status — Values: draft, pending, approved, rejected.
- course_level — Values: beginner, intermediate, advanced.
- lesson_review_status — Values: pending, approved, rejected.

8.5 Key Indexes

- ix_users_email_lower — Case-insensitive email lookup (used for login).
- ix_users_rank_score — Partial index on rank_score DESC where role=student (leaderboard queries).
- ix_rank_entries_rank_score — Global leaderboard ordering.
- ix_rank_entries_state — State leaderboard filtering and ordering.
- ix_rank_entries_district — District leaderboard filtering and ordering.
- ix_audit_logs_created_at — Audit log pagination ordered by newest first.

9. Docker Setup

9.1 Services in docker-compose.yml

Service	Port	Description
rankscript_backend	8000	FastAPI with uvicorn. Hot-reload enabled in dev mode. Mounts ./backend as volume.
rankscript_frontend	3000	Next.js standalone server. Receives INTERNAL_API_URL=http://backend:8000 for rewrites.
rankscript_db	5432	PostgreSQL 15. Runs schema and seed SQL on first start. Health-checked before backend starts.
rankscript_redis	6379	Redis 7 with custom redis.conf. Health-checked.
rankscript_pgadmin	5050	Optional pgAdmin UI. Only starts with --profile tools flag.

9.2 How to Run the Project

Step 1 — Copy environment file

```
cp .env.example .env
```

Edit .env and set a strong SECRET_KEY value.

Step 2 — Build and start all services

```
docker-compose up --build
```

Step 3 — Access the application

- Frontend: http://localhost:3000
- Backend API docs: http://localhost:8000/docs
- pgAdmin (optional): http://localhost:5050

Step 4 — Force full rebuild (when files change)

```
docker-compose down
docker rmi rankscript-frontend rankscript-backend 2>/dev/null
docker-compose build --no-cache
docker-compose up
```

9.3 Network Configuration

- All containers share the rankscript_network Docker bridge network.
- The frontend container connects to the backend using the service name backend:8000.
- INTERNAL_API_URL=http://backend:8000 is set only for the frontend container.

- The browser only communicates with the frontend on port 3000. Port 8000 is never exposed to the browser.

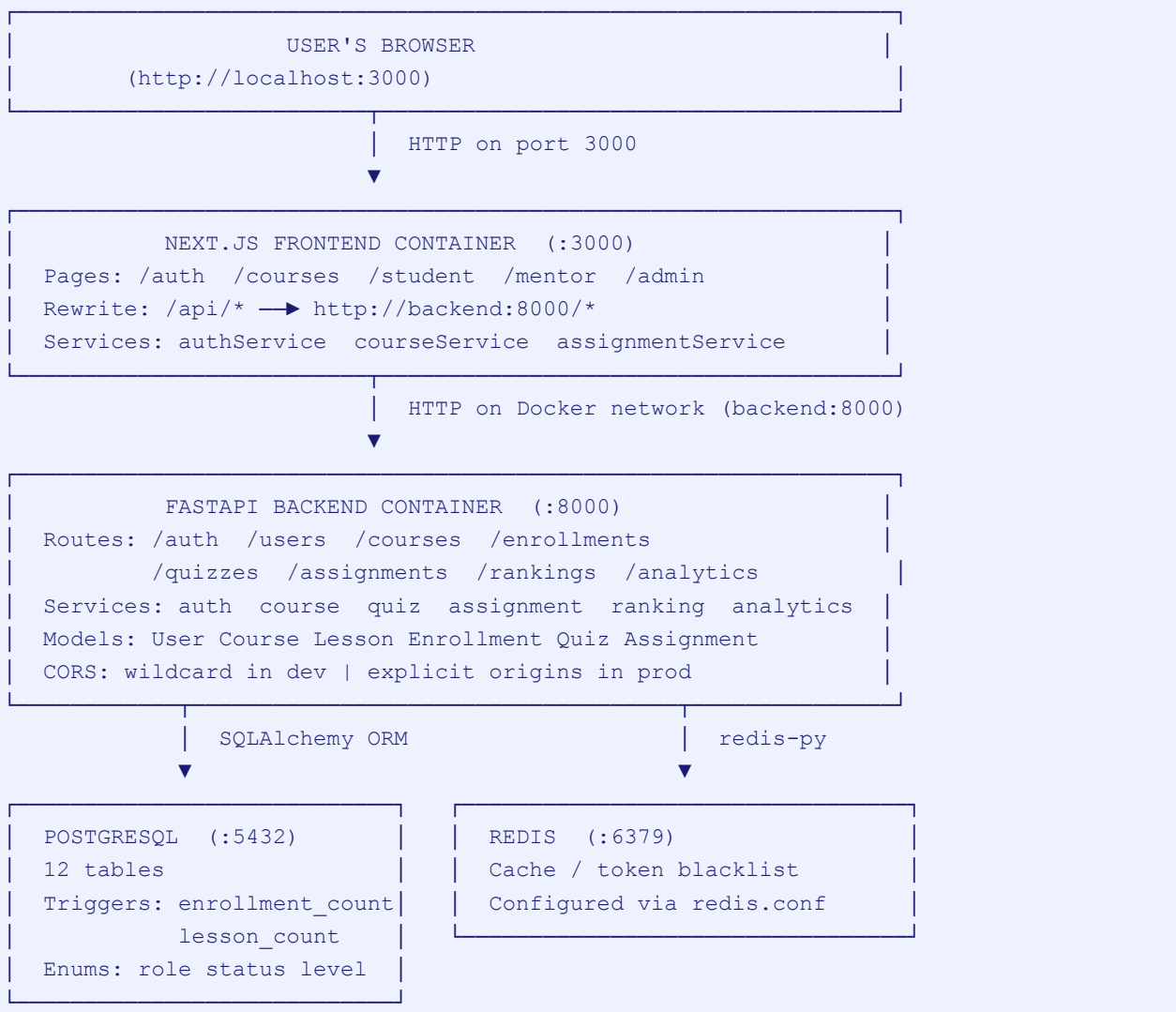
9.4 Seed Data

- On first start, PostgreSQL runs 00_schema.sql to create all tables, enums, indexes, and triggers.
- Then it runs seed_data.sql to create test accounts.
- Seed creates: 1 admin, 5 mentors, 20 students, 14 courses, 37 lessons, 10 quizzes, 4 assignments.
- All seed accounts use the password: password

Email	Role	Password
admin@test.com	admin	password
mentor.vineeth@test.com	mentor	password
student.roni@test.com	student	password

10. Diagrams

10.1 System Architecture Diagram



10.2 Data Flow Diagram — Assignment Submission

```
    │
    │  3. get_current_user() — validate JWT token
    │  4. Call submit_assignment() in assignment_service
    ▼
assignment_service.submit_assignment()
    │
    │  5. Check enrollment exists and is_approved = True
    │  6. Check no existing submission
    │  7. Check deadline — calculate late_days if late
    │  8. Create Submission row in PostgreSQL
    ▼
PostgreSQL
    │
    │  9. INSERT into submissions table
    ▼
FastAPI → Next.js → Browser
    │
    │  10. Return SubmissionResponse JSON
    │  11. Frontend renders 'Submitted' state
    ▼
Mentor Grades → ranking_service.update_user_ranking()
    │
    │  12. Recalculate rank_score
    │  13. Update rank_entries and users tables
```

11. Key Design Decisions

11.1 Why Microservice-Style Separation?

- Routes, services, models, and schemas are kept in separate layers.
- This makes each layer independently testable and replaceable.
- A route handler only delegates to a service. The service only knows about models and schemas.
- Bugs are easier to isolate. A database error appears only in the service layer.

11.2 Why Redis?

- Redis is included for production-ready token management.
- Refresh tokens can be blacklisted in Redis on logout, preventing token reuse.
- Redis is also used for caching leaderboard queries that run frequently.
- The infrastructure is already in place. Developers add Redis logic as the platform scales.

11.3 Why FastAPI?

- FastAPI generates API documentation automatically. Developers test endpoints at /docs without Postman.
- Pydantic integration means all input is validated before business logic runs.
- Python type hints make the code readable and IDE-friendly.
- Async support handles high concurrency without blocking threads.

11.4 Why Next.js?

- Server-side rewrites eliminate CORS issues entirely. The browser never contacts port 8000.
- File-based routing simplifies URL management — each folder is a page.
- React Context provides lightweight global state without Redux complexity.
- The standalone output mode creates a self-contained Docker image.

11.5 Why a Weighted Ranking Formula?

- A single metric (e.g. quiz score only) would not capture real learning.
- The formula balances four dimensions: knowledge (quiz), application (assignment), consistency (completion), and habit (streak).
- Weights can be adjusted without changing the database schema.

12. Important Notes

12.1 Environment Variables

Variable	Default	Purpose
DATABASE_URL	(required)	PostgreSQL connection string
SECRET_KEY	(required)	JWT signing secret. Must be at least 32 chars.
ENVIRONMENT	development	Controls CORS mode: development=wildcard, production=strict
NEXT_PUBLIC_API_URL	(empty)	Leave empty in Docker. Frontend uses /api prefix.
INTERNAL_API_URL	http://backend:8000	Docker-internal backend URL. Used by Next.js rewrites.
ACCESS_TOKEN_EXPIRE_MINUTES	30	JWT access token lifetime.
REFRESH_TOKEN_EXPIRE_DAYS	7	JWT refresh token lifetime.

12.2 Common Errors and Fixes

Error	Fix
Login failed / network error	Run: docker-compose build --no-cache. Check that NEXT_PUBLIC_API_URL is empty in docker-compose.yml.
CORS policy error	ENVIRONMENT must be 'development' in .env. Check backend logs for CORS mode.
GET /courses returns 405	Route was registered with '/' instead of ''. Apply the courses.py fix and rebuild.
Password verify fails	bcrypt version must be 3.2.2. Check backend/requirements.txt.
Frontend uses old code	Docker cached the build. Run: docker rmi rankscript-frontend then docker-compose build --no-cache.
ECONNREFUSED :8000	next.config.js still uses localhost:8000. It must use http://backend:8000 as the rewrite destination.

12.3 Debug Tips

- Check backend logs: docker logs rankscript_backend
- Check frontend logs: docker logs rankscript_frontend
- View API documentation: http://localhost:8000/docs
- Test CORS: http://localhost:8000/debug/cors
- Test health: http://localhost:8000/health

- Connect to database: psql -h localhost -U postgres -d rankscript -p 5432
- Access pgAdmin: Run docker-compose --profile tools up, then visit http://localhost:5050

End of Document